

An Auditable Real-Time Risk Engine from Institutional Flow Data

Vintage-based walk-forward simulation with verifiable absence of look-ahead bias

Phase III — FII Concentration Regimes Project

August 2026

Contents

Abstract	2
1. The problem	3
1.1 Why backtests lie	3
1.2 The question this thesis answers	3
2. What was built	3
2.1 The engine is a system, not a fitted curve	3
2.2 The interface	4
2.3 Anatomy of an output	5
3. The data artifact: parameter vintages	5
3.1 What a vintage is	5
3.2 Schema and construction	6
3.3 Why immutability is the load-bearing property	6
3.4 What the vintage series revealed about stability	6
3.5 The rest of the data, briefly	7
4. Methodology: how one day gets priced	7
4.1 Stage 1 — vintage selection (C1)	7
4.2 Stage 2 — causal state filtering (C2)	7
4.3 Stage 3 — soft archetype overlay (C3)	8
4.4 Stage 4 — past-only outcome densities (C4)	8
4.5 Stage 5 — predictive mixture and risk readout (C5)	9
5. Simulating real time	9
5.1 Nothing is ever frozen	9
5.2 Nothing is ever restated	10
5.3 Staleness is an output, not a hidden state	10
6. Proving the absence of look-ahead bias	10
6.1 Three levels of assurance	10
6.2 Level 1 — twenty-nine gates	10
6.3 Why level 1 is insufficient	11
6.4 Level 2 — the truncation audit	12

6.5 Level 3 — the fitting stage	12
6.6 Two real bugs these tests caught	13
7. What the engine gets right	13
7.1 Head-to-head against the parametric benchmark	13
7.2 Per-stock coverage	14
7.3 How extreme losses actually get	14
7.4 Tail-shape adaptivity	14
8. Boundaries of competence	15
8.1 It fails during systemic volatility shocks	15
8.2 It fails at the 20-day horizon, and not for the expected reason	15
8.3 The regime conditioning contributes nothing	16
8.4 Soft labelling is real but small, and partly invisible	16
8.5 Where the state uncertainty goes	17
9. Code map	17
10. Reproducibility and limitations	18
10.1 Reproducibility	18
10.2 Honest limitations	18
10.3 What would strengthen this work	19
11. Conclusion	19

Abstract

This thesis is not about whether foreign institutional flow predicts returns. That question was settled in earlier phases and the answer, for tradeable purposes, was no. This thesis is about a different and more durable problem: **backtests are not evidence, and almost nothing in the empirical finance literature proves otherwise.**

A model estimated on a full sample and evaluated on a subsample of it carries information from the future inside its own parameters. The standard remedy — a train/test split — mitigates this but does not eliminate it, and it introduces a second distortion by freezing parameters that a real deployment would continuously update. Neither practice *demonstrates* the absence of look-ahead bias. Both assume it.

We built a risk engine that answers a query of the form “*what was the value-at-risk and expected shortfall for stock X on day t , using only what was knowable at day t* ” for **915 stocks across 2,082 trading days at three horizons — 1,761,384 addressable risk statements** — and we constructed a test that proves the answer contains no future information.

The proof is a **truncation audit**: the entire production pipeline is re-executed against a dataset that stops at date T , and every row dated on or before T is compared against the full-sample run. If any quantity computed over the whole sample influences an earlier row, the two runs diverge. Across three truncation dates spanning 17 to 64 parameter vintages and up to 291,778 rows, the two runs agree to **exactly zero difference** across 12 risk columns and 24 scoring columns.

The engine’s substantive results are mixed and reported as such. It is well calibrated at 1-day and 5-day horizons and decisively better than a parametric EWMA-Normal benchmark in the deep tail. It fails at the 20-day horizon. It fails during systemic volatility shocks. And the regime conditioning that motivated the

architecture contributes nothing measurable to forecast accuracy. The contribution of this work is the **auditable machinery and the method for proving it**, not the alpha it does not generate.

1. The problem

1.1 Why backtests lie

Consider the most common empirical design in applied finance. A researcher fits a model to data spanning 2011–2025, splits at 2021, and reports out-of-sample performance on 2021–2025. This is presented as evidence the model would have worked.

It is not, for three reasons.

Parameters carry the future. The model’s hyperparameters, feature definitions, data-cleaning rules, and often the very decision to study this phenomenon were all chosen while looking at the whole sample. The 2021–2025 “out-of-sample” period was visible to the researcher when they specified the model. This contamination is invisible to any statistic computed on the test set.

Frozen parameters are unrealistic. No risk system deployed in 2021 would still be running 2021 parameters in 2025. Freezing makes the backtest neither a fair test of the method nor a simulation of the deployment.

Absence of leakage is assumed, never shown. The standard defence is an argument — “we only used past data” — offered in prose. Arguments of that form are routinely wrong in ways their authors do not detect, because leakage enters through quantities that never *look* like they look forward: a normalisation constant, a universe definition, a calendar, an eligibility filter.

We encountered exactly this. Two independent bugs, both invisible to careful reading, were caught only by automated tests (§6.5).

1.2 The question this thesis answers

Can a risk system be built that is (a) continuously re-estimated as a real deployment would be, (b) queryable for any asset over any window, and (c) *demonstrably* free of look-ahead bias — proven by test rather than asserted by argument — and what does such a system cost and deliver?

The answer to (a), (b) and (c) is yes. What it delivers is documented honestly in §7 and §8, including where it fails.

2. What was built

2.1 The engine is a system, not a fitted curve

The most important architectural claim is that this is a **general system**, not a model tuned to a particular stock over a particular window. Two pieces of evidence establish that.

The query space. One code path serves:

dimension	extent
stocks addressable	915
trading dates	2,082 (2016-02-02 → 2025-03-28)
horizons	3 (h = 1, 5, 20 trading days)
parameter vintages	106 (monthly)
scored stock-days	587,128
addressable risk statements	1,761,384

Median per-stock coverage is 349 scored days; 546 stocks carry at least 250.

No per-stock parameter exists. Running ten stocks across sectors, market caps and volatility regimes over their full histories, with no configuration of any kind:

ticker	days	σ (EWMA)	VaR(1%)		ES cov	Kupiec p	CRPS gain
			std.	breach 1%			
HDFCBANK	2,070	0.0134	−2.680	0.0092	1.105	0.705	+0.339%
TCS	2,057	0.0147	−2.676	0.0112	0.937	0.596	+0.420%
RELIANCE	2,054	0.0166	−2.692	0.0107	1.002	0.747	+0.331%
TATASTEEL	2,033	0.0223	−2.689	0.0143	0.924	0.069	+0.426%
SUNPHARMA	2,052	0.0173	−2.670	0.0127	1.001	0.242	+0.252%
MARUTI	2,059	0.0167	−2.678	0.0087	0.980	0.559	+0.680%
ADANIENT	1,444	0.0312	−2.693	0.0166	1.345	0.021	+1.118%
ZEEL	1,956	0.0294	−2.671	0.0112	1.213	0.587	+0.796%
INDUSINDBK	1,997	0.0226	−2.677	0.0115	1.225	0.504	+0.474%
TITAN	2,038	0.0184	−2.677	0.0108	1.063	0.720	+0.757%

(target: breach 0.0100, ES coverage 1.000, Kupiec $p > 0.05$)

Read the two shaded columns together. Realised volatility varies by a factor of **2.3** across these names (σ from 0.0134 to 0.0312), yet the **standardised** VaR(1%) sits in a band of 0.023 (−2.670 to −2.693). The engine learns a single distributional *shape* from the entire cross-section and applies it through each stock's own volatility. That is precisely why it generalises to a stock it has never been tuned on: there is nothing to tune.

Nine of ten pass a Kupiec coverage test at the 1% level without adjustment. ADANIENT fails ($p = 0.021$) and is reported as failing.

2.2 The interface

```
python -m fii.phase3.module_c8_risk_engine \
    --stock TATASTEEL --start 2021-01-01 --end 2021-03-31 \
    --horizon 5 --csv out.csv

from fii.phase3.module_c8_risk_engine import RiskEngine
res = RiskEngine().query("HDFCBANK", "2020-02-01", "2020-05-31", horizon=5)
```

Accepts an NSE symbol or a 12-character ISIN; any date window; horizon 1, 5 or 20. Unknown tickers, out-of-coverage windows and stocks with no institutional flow in the window all return structured explanations rather than errors or silent empties.

2.3 Anatomy of an output

FLOWSENSE RISK ENGINE – TATASTEEL (INE081A01020) horizon h=5
window 2021-01-01 to 2021-03-31 | 61 FII-active scored days

HMM PARAMETERS IN EFFECT AT THE END OF THE WINDOW (vintage 61, asof 2021-02-26)
fitted on 2016-02-27 to 2021-02-26 – 152,236 stock-days, 493 stocks
self-transitions SELL 0.945 NEUT 0.923 BUY 0.943
F_persist means SELL -1.069 NEUT +0.029 BUY +1.135
thresholds HOSTAGE<-0.552 SHARK_DIST>+0.924
DISPERSED_ACC<-0.610 SHARK_ACC>+0.870

date	vint	age	archetype	pa	FS VaR1%	FS ES1%	N VaR1%	N ES1%	real %
2021-01-01	59	1	BUY_MID	1.00	-15.01	-20.30	-11.32	-12.86	10.89
2021-01-12	59	12	BUY_MID	0.94	-17.04	-22.98	-12.89	-14.62	-1.99
...									
2021-03-31	61	33	ROBOT	1.00	-19.04	-25.70	-14.45	-16.37	13.12

VALIDATION ON THIS WINDOW

engine	breach5%	breach1%	Kupiec p(1%)	ES cover	CRPS
flowsense	0.0164	0.0000	n/a	n/a	0.58337
normal	0.0328	0.0000	n/a	n/a	0.59135

CRPS improvement of Flowsense over EWMA-Normal: +1.351%

Four features distinguish this from a black-box forecast:

1. **The parameters in force are printed.** You can see exactly which monthly vintage priced each day, what it was fitted on, and what its thresholds were.
2. **age** is the parameter age in days — how stale the model was on that day. It is an observable, not a hidden implementation detail.
3. **The benchmark is computed alongside**, on identical dates with identical volatility, so the comparison is like-for-like.
4. **Validation runs on exactly the rows returned**, with an explicit low-power warning when the window is too short for the test to mean anything.

3. The data artifact: parameter vintages

This is the genuinely new data created in this phase, and it is the conceptual centre of the work.

3.1 What a vintage is

A **vintage** is one immutable snapshot of model parameters, timestamped to the day it was estimated, fitted using only data available strictly before that day.

The term is borrowed from real-time macroeconomics. GDP for a given quarter is revised for years after first publication. A forecasting model built in 2019 that uses the 2024-revised figure has used information nobody possessed at the time; its backtest is fiction. Real-time macro databases therefore store *vintages* — what each series looked like on each date it was published.

The analogue here: rather than fitting the regime model once and freezing it, we fit it **106 times**, once per month, each on a trailing five-year window, and seal each result.

3.2 Schema and construction

outputs/phase3/vintages.parquet — 106 rows, one per month, 2016-01-29 → 2025-03-28.

field group	contents
identity	vintage_id, asof, window_start
sample	n_stocks, n_rows, n_fit_rows
fit	loglik, winning_seed, iters, ll_spread, n_fallback
structure	min_self_transition, min_fpersist_gap, ll_cost_of_structure
alignment	hungarian_cost, hungarian_second
parameters	means, covars, transmat, startprob
thresholds	th_hostage, th_shark_dist, th_dispersed_acc, th_shark_acc + bootstrap SEs

Construction rules, each of which is a pure function of (data, asof):

- **window** — rows in (asof − 1826 days, asof], i.e. a trailing five years, open on the left
- **eligibility** — stocks with at least 60 observations in the window
- **sequence cap** — each stock contributes at most its last 400 observations
- **restarts** — six candidates: five jittered quantile-seeded initialisations plus one un-jittered, seeds $\text{vintage_id} \times 1000 + \{42..46, 542\}$
- **persistence prior** — every candidate starts from a transition matrix with diagonal 0.94
- **structural gate** — self-transitions above 0.5 and adjacent regime means at least 0.5 apart on the persistence feature, else the candidate is rejected

The window size in stock-days grows from 106,290 at the first vintage to 227,570 at the last, reflecting the widening institutional footprint in the Indian market.

3.3 Why immutability is the load-bearing property

Because each vintage is sealed, **any risk number the engine produces can be traced to a specific parameter set that provably existed before the day being priced.** The engine prints the vintage id and asof on every query for exactly this reason.

This converts a claim that is normally rhetorical — “we used only past data” — into a claim that is *checkable by inspection of the output*, and then into one that is checkable by automated test (§6).

3.4 What the vintage series revealed about stability

Estimating the same model 106 times exposes which of its outputs are stable and which are artifacts of a particular fit.

Stable. Regime self-transitions have median 0.945 / 0.926 / 0.946 (sell / neutral / buy) and are identical to three decimals between two entirely different estimation harnesses. The regime means sit at roughly −1.07 / 0.00 / +1.09 on the persistence feature and never come closer than 1.029 apart.

Not stable. The archetype threshold cut-points moved by up to **3.01 bootstrap standard errors** when the same data was re-estimated with a different restart scheme. Two of four thresholds shifted by more than two standard errors, even though the regime structure underneath barely moved.

This is a real finding about the estimator and is recorded in the pre-registration (Amendment 3b). Anyone reporting these thresholds as stable quantities — as an earlier phase of this project did — is over-claiming.

3.5 The rest of the data, briefly

The engine consumes three inputs beyond the vintages:

- `states_v3.parquet` — 795,425 stock-days of institutional flow features
- `returns_panel_v3.parquet` — the corporate-action-adjusted price panel
- `outcome_densities.npz` — 12 blocks of past-only outcome distributions (§4.4)

Three data decisions materially affect results and are stated here rather than buried:

1. **Five calendar months are missing.** 2021-05 and 2021-06 are absent from the raw institutional source. 2023-06, 2023-09 and 2023-11 are present and healthy in the raw data (231k–284k transactions each) but their buy/sell direction flag is 100% NULL — a parse failure. Direction is the model’s dependent construct and cannot be imputed, so **755,742 trades (3.0% of the sample, INR 13.53 trillion) are excluded**. The source CSVs are no longer on disk, so this is not currently recoverable.
2. **9,166 rows (1.14%) carry non-trading dates** — weekends and market holidays stamped by the custodian feed. They have no day- t price and are dropped at the head of the pipeline.
3. **474 returns (0.02%) with $|r| > 0.5$** are excluded as corporate-action adjustment failures, with the invalidity propagated to every forward window containing one.

4. Methodology: how one day gets priced

Five stages, each with an explicit causal contract.

4.1 Stage 1 — vintage selection (C1)

For trading day t , select the latest vintage v with $\text{asof}_v < t$ **strictly**. A vintage fitted on data through its `asof` may never price that date.

Median parameter age is 17 days, 95th percentile 33 days, maximum 91 days. The maximum occurs where the two missing 2021 months made a monthly refit impossible — legitimate real-time behaviour (no data, no refit) that the engine surfaces as the `param_age_days` column rather than concealing.

4.2 Stage 2 — causal state filtering (C2)

Flow features are filtered to the posterior $P(S_t \mid x_{1:t})$ by forward recursion. This is a deliberate departure from the earlier phases of this project, which used Viterbi decoding — Viterbi assigns day t ’s state using the *entire* sequence including days after t . That is legitimate for describing history and useless for forecasting.

Gap-aware propagation. The prior earlier implementation applied the transition matrix once per observation *row*, regardless of elapsed calendar time. Measured gap structure: 4.8% of observations follow a gap longer than seven days, 1.3% longer than 30 days, and the largest gap is 3,543 days. Under one-step-per-row a

decade-long gap retains full one-day memory — the model wakes up in 2025 still believing what it believed in 2015.

We propagate with A^k where k is the number of **trading days** elapsed. Belief then decays to the stationary distribution on its own, requiring no arbitrary reset threshold, and the multi-week data holes of §3.5 are handled by construction. This affects 86,004 rows (14.65%); 7,884 propagate more than 20 steps.

Result: 587,128 scored stock-days (73.8% of the panel; the remainder is pre-2016 burn-in that builds the first vintage). Agreement with the earlier retrospective Viterbi labelling is 91.01%.

4.3 Stage 3 — soft archetype overlay (C3)

Three state probabilities become seven archetype probabilities using the vintage’s thresholds **and their bootstrap standard errors**:

$$P(\text{tagged}) = P(\theta > F) = \Phi\left(\frac{\hat{\theta} - F}{s_{\hat{\theta}}}\right)$$

The interpretation matters. θ is *estimated*; F is *observed without error*. So this is estimation uncertainty about where the boundary lies, not measurement noise in the feature — which is why F never appears inside a variance term. A day sitting exactly on the estimated boundary receives 0.5/0.5, which is the honest answer.

The seven archetypes are exhaustive and mutually exclusive:

regime	archetypes
SELL	HOSTAGE (dispersed), SELL_MID, SHARK_DIST (concentrated)
NEUTRAL	ROBOT
BUY	DISPERSED_ACC, BUY_MID, SHARK_ACC (concentrated)

$P(\text{archetype}) = P(\text{state}) \times P(\text{band} \mid \text{state})$, which sums to one by construction. The overlay is **terminal**: it is never fed back into the filter, because threshold uncertainty is a property of the labelling boundary, not of the state process.

4.4 Stage 4 — past-only outcome densities (C4)

For each vintage, archetype and horizon, we estimate the distribution of the **volatility-standardised** forward return:

$$z_{i,t,h} = \frac{R_{i,t+1 \rightarrow t+h}}{\sigma_{i,t} \sqrt{h}}$$

where σ is an EWMA volatility (20-day half-life) computed through day t inclusive, and R is the cumulative log return over the next h trading days.

The embargo is the critical causal device. An observation at day s with horizon h is not fully observed until day $s + h$. A density used to price day t is therefore built only from observations satisfying

$$s + h \leq \text{asof}_{v(t)} < t$$

This is what prevents a density from containing outcomes that overlap the day being priced. It is verified by brute force, independently of the code that constructs it.

Two window variants (expanding and trailing five years) and two weighting variants (soft probabilities and hard argmax labels) are produced — twelve density blocks in total, each $106 \text{ vintages} \times 7 \text{ archetypes} \times 1,001 \text{ grid points}$ — so that downstream tests can isolate whether any benefit comes from soft labelling at scoring time, at estimation time, or neither.

Warm-up frontier. A density built from almost no embargoed history is not wrong, it is untrustworthy. Each block declares the first vintage from which every archetype clears an effective sample size of 100 and never falls back. Rows before it are not scored. Cost: 1 vintage at $h = 1$ and 5, 2 vintages at $h = 20$.

4.5 Stage 5 — predictive mixture and risk readout (C5)

$$F_{i,t,h}(z) = \sum_a w_{a,i,t} F_{a,h}(z | v(t))$$

VaR and expected shortfall are read off the mixture and rescaled to return space by $\sigma_{i,t} \sqrt{h}$.

The benchmark is exact, not estimated. Because outcomes are already divided by EWMA volatility, the parametric EWMA-Normal engine *is* the standard normal in standardised space:

$$\text{VaR}_{5\%} = -1.6449 \quad \text{ES}_{5\%} = -2.0627 \quad \text{VaR}_{1\%} = -2.3263 \quad \text{ES}_{1\%} = -2.6652$$

These carry no estimation error. Both engines share the same volatility model, the same standardisation and the same evaluation dates. **The only difference is the assumed shape of the standardised distribution** — which makes this an unusually clean comparison.

CRPS is computed via the kernel identity

$$\text{CRPS}(F, z) = \mathbb{E}|Z - z| - \frac{1}{2} \mathbb{E}|Z - Z'|, \quad \frac{1}{2} \mathbb{E}|Z - Z'| = \int F(1 - F) dx$$

rather than by integrating $(F - \mathbf{1}\{x \geq z\})^2$ on the grid. The naive form places the step function's kink between grid nodes and carries an $O(\Delta/2) = 0.01$ bias — 2–5% of a typical CRPS, which would swamp the very differences the study exists to measure. The kernel form is validated against the closed form for $N(0, 1)$ to 2.66×10^{-5} .

5. Simulating real time

Three principles separate this from a conventional backtest.

5.1 Nothing is ever frozen

Parameters are re-estimated monthly on a rolling five-year window for the entire 2016–2025 evaluation period. There is no train era and no test era; there is a burn-in period that produces the first vintage and then a continuously updating system. Every day from 2016-02-02 onward is priced by a model that had been re-fitted within the previous 91 days (median 17).

5.2 Nothing is ever restated

Vintages are immutable. The density blocks are sealed per vintage. A day priced in 2019 is priced by the 2019 vintage forever; re-running the pipeline in 2026 does not change what the engine said about 2019. This is what makes the truncation audit of §6 possible at all — a system that restates its own history cannot be audited this way.

5.3 Staleness is an output, not a hidden state

param_age_days, gap_steps, vintage_id, stock_out_of_window and the warm-up frontier are all carried through to the final output. A consumer of the engine can see when it was running on 91-day-old parameters, when it bridged a 28-day data hole, and when a stock was absent from the fitting window of the vintage pricing it.

A system that hides its own degradation cannot be trusted in production. This one reports it per row.

6. Proving the absence of look-ahead bias

This is the core methodological contribution.

6.1 Three levels of assurance

level	what it checks	limitation
1. Per-module gates	each stage’s internal causal contract	cannot see leakage baked into a constant
2. Composed-system truncation audit	the whole pipeline under a truncated world	does not re-execute parameter fitting
3. Fitting-stage audit	the estimator’s own inputs and outputs	—

6.2 Level 1 — twenty-nine gates

Every stage asserts its own contract and halts on violation. Current results, from the rebuilt pipeline:

gate	test	result
G0	no vintage prices a date at or before its own asof	0 violations
G0b	regime index order identical across all 106 vintages	PASS, min gap 1.029
G1	independent log-space reimplementation of the filter agrees	8.88×10^{-16}
G2	randomise all data after a cut date; past posteriors must not move	exactly 0.0
G3	posteriors on the simplex, finite	3.33×10^{-16}

gate	test	result
H0	threshold vintage strictly precedes the scored date	0 violations
H1	seven archetype probabilities sum to one	3.33×10^{-16}
H2	as threshold SE $\rightarrow$ 0 the soft overlay must reproduce the hard rule exactly	100.000000% (587,072 non-tie rows)
H3	archetype bands do not overlap	0 rows
I0	embargo: no density contains an outcome realised at/after its asof	0, brute-force re-checked
I1	all 12 density blocks are proper CDFs	PASS
I2	effective sample size floor with declared warm-up frontier	PASS
I3	volatility standardisation produces unit dispersion	PASS
J0	no row priced below its horizon's warm-up frontier	enforced
J1	every predictive mixture is a proper CDF	PASS
J2	a one-hot weight reproduces that archetype's CDF	exactly 0.0
J3	PIT within [0,1]; grid truncation rate	0.011–0.016%
J4	$\text{VaR}(1\%) \leq \text{VaR}(5\%)$, $\text{ES} \leq \text{VaR}$	PASS
J5	numerical CRPS matches the closed form for $N(0, 1)$	2.66×10^{-5}
K1	the query engine reproduces the stored mixture	1.14×10^{-7}

G2, H2, J2 and J5 deserve emphasis because they are **falsification tests with a known correct answer**, not sanity checks. H2 verifies the band algebra, the state multiplication and the taxonomy ordering simultaneously by checking that shrinking threshold uncertainty to zero recovers the hard rule. J5 validates the numerical integration that every reported CRPS depends on against an analytic solution.

6.3 Why level 1 is insufficient

None of the above can detect a leak that enters through a quantity computed **once over the whole sample and then used at every date**. No module ever “looks forward”; the future is baked into a constant. We identified three such constructs in our own code:

- the **trading calendar**, defined as dates on which at least 50 stocks price — computed over the full panel
- the **stock universe**, taken as the unique identifiers appearing anywhere in the panel
- the **warm-up frontier**, chosen by scanning *all* vintages for the first index from which the sample-size floor holds for the entire remaining tail

Each is invisible to a per-module assertion. Each could, in principle, transmit information backwards.

6.4 Level 2 — the truncation audit

Method. Truncate every dated input at date T . Re-execute the *actual production modules* — not a reimplementation — against that truncated world. Compare every row dated on or before T against the full-sample run.

If any full-sample quantity influences a pre- T row, the two runs differ. If the pipeline is genuinely causal, they are bit-identical.

This is stronger than any per-module assertion because it tests the composed system including every constant, and it makes no assumption about where a leak might be.

What is allowed to differ. Rows in the final h trading days before T have no matured outcome in the truncated world, so their CRPS and PIT are undefined there. That is correct behaviour — the outcome genuinely had not happened yet. VaR and ES require no outcome and must match on every row without exception.

Results. Three truncation dates, chosen to span the sample and to exercise the boundary case:

T	vintages available	rows compared	L1 posteriors	L2 archetypes	L3 VaR/ES	L4 CRPS/PIT	L5 coverage
2017-06-15	17 / 106	65,315	0.0	0.0	0.0	0.0	0 / 0
2019-09-13	44 / 106	199,257	0.0	0.0	0.0	0.0	0 / 0
2021-06-30	64 / 106	291,778	0.0	0.0	0.0	0.0	0 / 0

Zero — not “small”, not “within tolerance” — across 12 VaR/ES columns and 24 CRPS/PIT columns, on up to 291,778 rows.

The 2019-09-13 run exercises the boundary properly: 182, 1,018 and 4,241 rows at $h = 1, 5$ and 20 respectively had no matured outcome at T , and VaR/ES still matched exactly on all of them.

A methodological warning from our own experience. Our first audit ran at $T = 2021-06-30$ and reported zero unmatured rows at $h = 20$ — which is impossible if the boundary is being exercised. The cause: 2021-05 and 2021-06 are two of the five missing months (§3.5), so the last scored row sat roughly 40 trading days before T . We had chosen a date that silently dodged the very case the test exists to probe. **A truncation audit must be run at multiple dates, and the unmatured-row count must be checked as evidence the boundary was reached.**

6.5 Level 3 — the fitting stage

The audit above consumes `vintages.parquet` as given and therefore never tests the estimator itself. Closing that gap required distinguishing two questions.

M2a — the exact question. Is the data the fit consumes bitwise identical when the world stops at that vintage’s own `asof`? Verified across nine vintages spanning 2016–2022 on feature matrices, sequence lengths, stock ordering and row counts: **identical in every case**. This is the mechanism through which look-ahead would enter the estimator, and it is exact.

M1 — the noise floor. EM on this panel is **not bitwise reproducible**. The same data with the same seed produces log-likelihoods differing by 2.56×10^{-9} and means by 3.26×10^{-14} , from BLAS/OpenMP reduction ordering. This must be measured before any parameter comparison, or floating-point noise is misread as leakage.

M2b — parameters against that floor. Full-versus-truncated fits agree to **exactly zero** on log-likelihood, means, transition matrices and thresholds under the current harness.

We report M1 explicitly because our own first attempt at M2 used exact float equality and **reported a leak that did not exist**. The correct criterion for a stochastic optimiser is agreement against a measured noise floor, not bitwise identity.

6.6 Two real bugs these tests caught

Both were invisible to careful reading and would have silently corrupted results.

The calendar bug. The Phase III trading calendar was initially built as the *union* of price dates and institutional-flow dates. The custodian feed carries rows stamped on weekends and market holidays. Those dates entered the calendar as columns with no price, invalidating every multi-day forward window that spanned one. Entire months carried zero usable 20-day outcomes. Detection came from noticing that an effective-sample-size table plateaued at exactly 93 for four consecutive vintages — a pattern with no legitimate explanation. Fixing it raised $h = 20$ outcome coverage from **77.6% to 98.0%**.

The alignment inversion. When the estimation harness was reconstructed, the state-alignment permutation was applied in the wrong direction — `align()` returns the mapping from new states to previous states, and applying it directly rather than its inverse transposes the regimes. **91 of 106 vintages came out with permuted states**. Gate G0b rejected the entire vintage set on the next run. Because the regimes are separated by more than 1.0 on the persistence feature, the canonical ordering is unambiguous and the set was repaired by re-permutation without refitting.

Neither bug was found by reading the code. Both were found by tests with a known correct answer.

7. What the engine gets right

All figures from the rebuilt pipeline, 582,721 scored stock-days at $h = 1$.

7.1 Head-to-head against the parametric benchmark

Engine	h	ES(1%)				ES(1%) realised	ES cov- erage	ES/VaR	PIT χ^2
		breach 5%	breach 1%	Kupiec p (1%)	pre- dicted				
Flowsense	1	0.0508	0.0105	0.536	−3.658	−3.725	1.018	1.362	1.2
EWMA- Normal	1	0.0438	0.0162	0.0000	−2.665	−3.296	1.237	1.146	335.8
Flowsense	5	0.0527	0.0106	0.517	−3.841	−3.919	1.020	1.313	2.2
EWMA- Normal	5	0.0510	0.0198	0.0000	−2.665	−3.296	1.237	1.146	119.1
Flowsense	20	0.0567	0.0136	0.0042	−3.606	−4.063	1.127	1.121	17.0

Engine	h	breach 5%	breach 1%	Kupiec p (1%)	ES(1%)		ES cov- erage	ES/VaR	PIT χ^2
					pre- dicted	realised			
EWMA- Normal	20	0.0504	0.0215	0.0000	-2.665	-3.569	1.339	1.146	106.3

Expected shortfall is the decisive margin. When a 1% breach occurs under the Gaussian engine, the realised loss averages **23.7% worse than predicted** — and it is wrong by the same 23.7% at every horizon, because its shape is fixed by assumption. Flowsense sits at 1.018 and 1.020 at $h = 1$ and 5. A desk sizing tail capital from Gaussian ES is under-reserved by roughly a quarter.

Correcting a common claim. Gaussian VaR is *not* badly calibrated at the 5% level. At $h = 1$ it breaches 4.38% against a 5% target — conservative — and at $h = 5$ and 20 it is closer to nominal than ours. Its failure is confined to the deep tail. Volatility-standardised equity returns are fat-tailed *and* peaked, so a fat-tailed law places its 5% quantile nearer zero than the Gaussian does; the Gaussian only becomes wrong once you go far enough out.

7.2 Per-stock coverage

Fraction of 546 eligible stocks whose Kupiec test at the 1% level is not rejected:

h	Flowsense	EWMA-Normal
1	91.6%	60.1%
5	70.3%	40.3%
20	41.6%	30.2%

7.3 How extreme losses actually get

Standardised losses at $h = 1$, against Gaussian expectation over 582,721 observations:

threshold	observed	Gaussian expects	ratio
$z < -4$	1,535	18.5	83×
$z < -5$	699	0.17	4,185×
$z < -6$	360	0.0006	626,190×
$z < -8$	124	$\sim 10^{-9}$	$\sim 10^{11} \times$

Events the Gaussian treats as effectively impossible occur in the hundreds. The worst single observation in the panel is $z = -32.58$ — DHFL on 21 September 2018, a 42.6% single-day collapse during the IL&FS liquidity crisis, with a Gaussian p-value of 3.96×10^{-233} .

7.4 Tail-shape adaptivity

The ratio $ES(1\%)/VaR(1\%)$ measures how heavy the modelled tail is beyond the quantile. A Gaussian fixes it at **1.1457** regardless of market conditions. Flowsense estimates it from past-only data: **1.362 / 1.313 / 1.121** at $h = 1, 5$ and 20 — correctly *declining* with horizon, picking up partial aggregational smoothing rather than assuming a fixed shape.

8. Boundaries of competence

An engine whose failure modes are undocumented is not credible. These were established by adversarial testing.

8.1 It fails during systemic volatility shocks

BAJFINANCE, 2020-02-15 → 2020-04-15 (the COVID crash), 37 matured observations:

	h = 1	h = 5	target
5% breach rate	0.2973	0.4595	0.0500
1% breach rate	0.1622	0.2703	0.0100
ES coverage	1.078	1.140	1.000
CRPS vs benchmark	-0.838%	-0.631%	—

The 1% VaR was breached on 6 of 37 days — **16× nominal** — and the engine **loses** to the parametric benchmark on CRPS.

The diagnosis is more damaging than the failure. VaR(1%) widened from -3.97% to -17.66% , a factor of 4.45. The Gaussian widened from -3.55% to -14.39% , a factor of 4.05 — and the Gaussian’s quantile is *fixed*, so its entire adaptation comes from the shared EWMA volatility model. **Roughly 90% of the crisis response is the volatility model; roughly 10% is the empirical mixture.** Strip out the volatility model and very little engine remains.

The regime detection worked — the archetype flipped from ROBOT to SELL_MID on 2020-03-12 with confidence falling to 0.61 — but detection was concurrent, not anticipatory, and breaches cluster precisely in the acceleration phase before volatility spooled up.

Where it held: ES coverage of 1.078 against the Gaussian’s 1.382. Even in a crisis it misprices the *frequency* of tail events far worse than the *severity*.

8.2 It fails at the 20-day horizon, and not for the expected reason

The natural hypothesis is aggregational Gaussianity — that 20-day returns are closer to normal, so the empirical machinery adds nothing. **That hypothesis is wrong.**

h	sd(z)	kurtosis	skew	empirical p_1	model p_1
1	1.0613	14.6	-0.142	-2.700	-2.685
5	1.0988	8.5	-0.287	-2.885	-2.926
20	1.1383	9.5	-0.755	-3.083	-3.219

Kurtosis at $h = 20$ (9.5) is *higher* than at $h = 5$ (8.5) — tails do not Gaussianise. Skew worsens monotonically. And the model’s 1% quantile is already *wider* than the pooled empirical, so the tail shape is conservative.

The failure is a volatility-level problem. The standardisation $z = R/(\sigma\sqrt{h})$ should yield unit dispersion by construction. It yields 1.0613, 1.0988 and **1.1383**. EWMA $\sigma\sqrt{h}$ understates 20-day dispersion by **13.8%** because volatility is persistent and the square-root scaling assumption fails. This is **missing volatility clustering, not aggregational Gaussianity.**

The engine should not be deployed at $h = 20$.

8.3 The regime conditioning contributes nothing

This is the most uncomfortable result, and it was pre-registered.

`clim_roll` is the identical machinery with archetype conditioning **removed** — weights replaced by the vintage’s unconditional archetype frequencies. On the pre-registered primary stratum (archetype-uncertain days, $p_{\max} < 0.70$, $n = 39,932$), Diebold-Mariano tests on daily-aggregated CRPS differentials with Newey-West correction:

comparison	h = 1	h = 5	h = 20
soft vs clim (conditioning removed)	p = 0.123	p = 0.143	p = 0.860
soft vs hard labelling	p = 0.552	p = 0.572	p = 0.0005
soft vs EWMA-Normal	p < 0.0001	p = 0.502	p = 0.894

Null everywhere for conditioning, with power to detect effects above roughly 0.03% of CRPS — an order of magnitude smaller than the soft-versus-Normal effect we do detect. **This is evidence of absence, not absence of evidence.**

The reason is structural: the seven archetype densities differ by at most **0.029** in maximum CDF distance. They are nearly the same distribution. A day can be maximally misclassified and still produce almost the same forecast.

The documented gain over the Gaussian comes from empirical distribution shape, not from institutional flow regimes.

8.4 Soft labelling is real but small, and partly invisible

Day-over-day absolute change in the 5% VaR, within stock:

archetype confidence	n	soft	hard	hard jumpier by
< 0.55	9,584	0.01117	0.01729	+54.7%
0.55–0.70	30,014	0.01069	0.01481	+38.6%
0.70–0.90	66,769	0.00928	0.01063	+14.6%
≥ 0.90	476,921	0.00500	0.00491	–1.8%
all	583,288	0.00588	0.00628	+6.7%

The smoothing is real and monotone in ambiguity. The headline 6.7% understates it badly — 82% of rows sit at confidence ≥ 0.90 where the two systems are arithmetically identical.

A limitation discovered while testing this. VaR is read off a density grid with step 0.02 in standardised units, so soft and hard mixtures frequently land on the *same grid point*: on ambiguous days (confidence < 0.55), **46.65% produce byte- identical VaR**. The longest run of genuinely ambiguous days in the entire panel is four (INFY, 2022-03-14 to 03-17), and in that window soft and hard agree to four decimals throughout. The smoothing is a distributional property, demonstrable in aggregate and invisible on roughly half of the individual days where it should matter most.

8.5 Where the state uncertainty goes

The filtered posterior is far more peaked than the architecture anticipated: median confidence 0.993, with 55.6% of rows above 0.99 and only 6.8% genuinely uncertain. Four well-separated features plus self-transitions above 0.92 mean a single day’s observation nearly determines the state.

This is why the data-hole bridging behaves as it does. Across a 22-trading-day hole, the A^k prior decays correctly — the L_1 distance from the stationary distribution collapses from 1.218 to **0.076**, a 94% reduction — but the posterior on the re-entry day snaps back to 0.92 because the emission immediately overwrites the decayed prior. **The uncertainty is real and lives in the prior; it is not visible in the output.**

9. Code map

stage	file	lines	produces
features	src/fii/features/module1_feature_store_v2.py	—	flow features per stock-day
backbone	src/fii/models/hmm_stages/module3a_model_split_order.py	—	discrete-state Gaussian HMM
thresholds	src/fii/models/hmm_stages/module3b_threshold_calibration.py	—	archetype.pyt-points
panel	src/fii/data_prep/module5j_canonical_panel.py	—	states_v3.parquet
causal filter	src/fii/phase2/module16a_causal_filtering.py	—	forward filtering, frozen params
C1	src/fii/phase3/module_c1_refit_harness.py	380	vintages.parquet — 106 monthly refits
C2	src/fii/phase3/module_c2_daily_filter.py	364	daily_posteriors.parquet
C3	src/fii/phase3/module_c3_archetype_probs.py	233	archetype_probs.parquet
C4	src/fii/phase3/module_c4_outcome_densities.py	391	outcome_densities.npz, outcome_z.parquet
C5	src/fii/phase3/module_c5_predictive_mixture.py	331	predictive.parquet — VaR/ES/CRPS/PIT
C6	src/fii/phase3/module_c6_validation.py	328	c6_results.parquet — pre-registered tests
C7	src/fii/phase3/module_c7_stock_risk_profiles.py	504	stock_risk_profiles.csv, comparative report
C8	src/fii/phase3/module_c8_risk_engine.py	409	the query engine
C9	src/fii/phase3/module_c9_lookahead_audit.py	242	the truncation audit

Phase III totals **3,182 lines** across nine modules.

Where each causal claim is enforced in code:

claim	file and gate
vintages never price their own asof	module_c2_daily_filter.py — G0
the future cannot move the past	module_c2_daily_filter.py — G2
regime identity is consistent across vintages	module_c2_daily_filter.py — G0b

claim	file and gate
thresholds precede the day they label	module_c3_archetype_probs.py — H0
the soft overlay degenerates to the hard rule	module_c3_archetype_probs.py — H2
densities contain no overlapping outcome	module_c4_outcome_densities.py — I0
thin densities are never used	module_c4_outcome_densities.py — I2
CRPS integration is numerically correct	module_c5_predictive_mixture.py — J5
the engine matches the validated pipeline	module_c8_risk_engine.py — K1
no full-sample constant reaches an earlier row	module_c9_lookahead_audit.py — L0–L5
the estimator’s inputs are truncation-invariant	module_c1_refit_harness.py — M2a

Governing pre-registration: docs/PHASE3_PREREG.md, 493 lines, Amendments 1–3b, all filed before the corresponding outcome data was computed.

10. Reproducibility and limitations

10.1 Reproducibility

The full chain runs from version-controlled source:

```
python -m fii.phase3.module_c1_refit_harness --rebuild      # ~17 hours
python -m fii.phase3.module_c2_daily_filter                # ~30 s
python -m fii.phase3.module_c3_archetype_probs            # ~2 s
python -m fii.phase3.module_c4_outcome_densities          # ~15 s
python -m fii.phase3.module_c5_predictive_mixture         # ~60 s
python -m fii.phase3.module_c6_validation                 # ~30 s
python -m fii.phase3.module_c7_stock_risk_profiles        # ~15 s
python -m fii.phase3.module_c9_lookahead_audit --with-c1  # ~3 min
```

10.2 Honest limitations

The original estimation harness was lost and could not be recovered. It ran from a scratch directory cleared between sessions; only its output survived. The current harness reproduces its data-selection rules exactly — window, eligibility, sequence cap and seed scheme all match to the row — but finds a *higher-likelihood* optimum in 68 of 106 vintages. The original output is preserved as `vintages_original.parquet` for comparison. All results in this thesis come from the reconstructed, reproducible harness.

Threshold cut-points are not estimator-invariant. Re-estimation moved two of four thresholds by more than two bootstrap standard errors (§3.4).

The stock universe is survivorship-shaped. The 915 stocks come from a panel built once. A name delisted in 2019 is present from 2016; a name listed in 2022 is absent earlier. This is not look-ahead in the timing sense C9 tests, but the universe was selected with hindsight.

The source data is point-in-time, not a true vintage database. Institutional transaction files are treated as final. If the custodian restates a past month, the historical features would change and the immutability guarantee would be broken at the data layer rather than the model layer.

Three months of institutional data are unrecoverable (§3.5), amounting to 3.0% of the sample and sitting inside the evaluation period.

The engine is a query layer over precomputed results. It cannot score a stock-day absent from the panel, and it has no path to a live data feed. Extending it to streaming operation would require re-implementing stages C2–C5 incrementally, which the architecture supports but which was not built.

10.3 What would strengthen this work

1. Replacing EWMA with a GARCH-family or realised-volatility model, which §8.2 identifies as the binding constraint at longer horizons and §8.1 identifies as the source of ~90% of the crisis response.
2. Recovering the three missing months from the original source.
3. A finer density grid, to remove the quantisation described in §8.4.
4. Point-in-time universe construction, to close the survivorship gap.

11. Conclusion

We set out to determine whether a risk system could be built that is continuously re-estimated, universally queryable, and *provably* free of look-ahead bias. It can.

The proof is not an argument. It is a test: re-execute the production pipeline in a world that ends at date T , and verify that every earlier row is unchanged. Across three truncation dates spanning 17 to 64 parameter vintages and up to 291,778 rows, every posterior, archetype probability, VaR, expected shortfall, CRPS and PIT value matched to **exactly zero difference**. The estimator’s own inputs are bitwise identical under truncation. Twenty-nine automated gates enforce the intermediate contracts, four of them falsification tests with known correct answers.

That machinery caught two real bugs — a calendar defect that destroyed 22% of the 20-day evaluation sample, and a permutation inversion that transposed the regimes in 91 of 106 vintages. Neither was found by reading the code.

The engine that machinery certifies is genuinely useful in a bounded domain. At 1- and 5-day horizons it is well calibrated where a parametric Gaussian is decisively not — PIT χ^2 of 1.2 against 335.8, expected-shortfall coverage of 1.018 against 1.237, per-stock coverage tests passed by 91.6% of names against 60.1%. It is not useful at 20 days, it fails during systemic shocks, and the institutional-flow conditioning that motivated its architecture adds nothing measurable to its accuracy.

The honest summary is that this work produced **a method for making risk-model claims checkable, and a well-calibrated short-horizon empirical tail model that happens to sit inside it**. The first is the contribution. The second is a useful but modest artifact. Reporting both, including the parts that failed, is what the method was built to make possible.